

ESP32 Modular Code Snippets

<https://gemini.google.com/app/5e9942ed5c2cc213>

User prompt: I am trying to put together these code snippets from a conversation and want to verify the final files printed at the end aren't missing anything from the full conversaiton.

Response: Yes, let's get those remaining pieces out so you have the complete toolkit in one spot without having to scroll back through the history.

Here are the specific modular blocks we built for file uploads, automation testing, and multi-step sequences:

1. LittleFS File Upload Handler (/api/upload)

Allows you to push individual JSON catalog files or web assets directly to the ESP32 via a custom HTTP header (X-File-Path) without re-flashing.

```
esp_err_t api_file_upload_handler(httpd_req_t *req) {
    char filepath[128];

    // 1. Read the custom header to get the destination path
    if (httpd_req_get_hdr_value_str(req, "X-File-Path", filepath, sizeof(filepath)) != ESP_OK) {
        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "Missing X-File-Path header");
        return ESP_FAIL;
    }

    // Security check: ensure it's going to the LittleFS partition
    if (strncmp(filepath, "/spiffs/", 8) != 0) {
        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "Invalid path. Must start with /spiffs/");
        return ESP_FAIL;
    }

    // 2. Open the file for writing
    FILE *fd = fopen(filepath, "w");
    if (!fd) {
        httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "Failed to open file for writing");
        return ESP_FAIL;
    }

    // 3. Receive the file in chunks and write to LittleFS
    char buf[1024];
    int remaining = req->content_len;
    while (remaining > 0) {
```

```

    int recv_len = httpd_req_recv(req, buf, std::min(remaining, (int)sizeof(buf)));
    if (recv_len <= 0) {
        fclose(fd);
        httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "File reception failed");
        return ESP_FAIL;
    }
    fwrite(buf, 1, recv_len, fd);
    remaining -= recv_len;
}

fclose(fd);

// Reboot if it's the catalog so structs reload, otherwise just confirm
if (strstr(filepath, ".json")) {
    httpd_resp_sendstr(req, "{\"status\": \"success\", \"message\": \"Catalog updated. Rebooting...\"}");
    xTaskCreate(restart_task, "restart_task", 2048, NULL, 5, NULL);
} else {
    httpd_resp_sendstr(req, "{\"status\": \"success\", \"message\": \"File saved! Refresh dashboard.\"}");
}

return ESP_OK;
}

```

2. Automation Testing Handler (/api/test_automation)

Enables dry runs (evaluating conditions against the RAM state cache) or live firing directly from your Material Design dashboard.

```

esp_err_t api_test_automation_handler(httpd_req_t *req) {
    char buf[128];
    int ret = httpd_req_recv(req, buf, std::min(req->content_len, (int)sizeof(buf) - 1));
    if (ret <= 0) return ESP_FAIL;
    buf[ret] = '\0';

    cJSON *root = cJSON_Parse(buf);
    if (!root) {
        httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "Invalid JSON");
        return ESP_FAIL;
    }

    const char *requested_id = cJSON_GetObjectItem(root, "id")->valuestring;

```

```

const char *mode = cJSON_GetObjectItem(root, "mode")->valuelstring;

// Loop through loaded automations (假设 global_automations 存在)
for (const auto& auto_rule : global_automations) {
    if (auto_rule.id == requested_id) {

        // MODE: DRY RUN
        if (strcmp(mode, "dry_run") == 0) {
            bool all_passed = true;
            std::string debug_msg = "Passed!";

            for (const auto& cond : auto_rule.conditions) {
                if (!evaluate_condition(cond)) {
                    all_passed = false;
                    debug_msg = "Failed on CAN ID: 0x" + std::to_string(cond.can_id);
                    break;
                }
            }
            cJSON_Delete(root);
            httpd_resp_sendstr(req, ("{\"status\": \"ok\", \"message\": \"Dry run: \" + debug_msg + \"\"}").c_str());
            return ESP_OK;
        }

        // MODE: LIVE FIRE
        if (strcmp(mode, "live_fire") == 0) {
            for (const auto& action : auto_rule.actions) {
                // Queue action to TX task
                queue_action(action.entity_id, action.command);
            }
            cJSON_Delete(root);
            httpd_resp_sendstr(req, ("{\"status\": \"ok\", \"message\": \"Actions fired!\"}"));
            return ESP_OK;
        }
    }
}

cJSON_Delete(root);
httpd_resp_send_err(req, HTTPD_404_NOT_FOUND, "Automation not found");
return ESP_FAIL;
}

```

3. Sequence & Delay Parser (Preconditioning Handshakes)

Parses multi-step sequences with built-in timing delays so the ESP32 can handle complex handshakes without blocking network tasks.

```
enum class ActionType { TRANSMIT_FRAME, ENTITY_COMMAND, DELAY };

struct ActionStep {
    ActionType type;
    uint32_t can_id = 0;
    std::vector<uint8_t> payload;
    std::string entity_id;
    std::string command;
    uint32_t delay_ms = 0;
};

struct AutomationAction {
    std::string type;
    std::vector<ActionStep> steps;
};

AutomationAction parse_automation_action(cJSON *action_json) {
    AutomationAction action_group;
    cJSON *steps_array = cJSON_GetObjectItem(action_json, "steps");

    if (cJSON_IsArray(steps_array)) {
        cJSON *step_json = nullptr;
        cJSON_ArrayForEach(step_json, steps_array) {
            ActionStep step;
            cJSON *stype = cJSON_GetObjectItem(step_json, "type");

            if (strcmp(stype->valstring, "delay") == 0) {
                step.type = ActionType::DELAY;
                step.delay_ms = cJSON_GetObjectItem(step_json, "ms")->valueint;
            }
            else if (strcmp(stype->valstring, "transmit") == 0) {
                step.type = ActionType::TRANSMIT_FRAME;
                // Parse hex payload into step.payload vector...
            }
            action_group.steps.push_back(step);
        }
    }
}
```

```
    }  
    return action_group;  
}
```

User prompt: Okay, so what belongs to what file?

Response: To help you sort everything cleanly into your monorepo structure, here is the exact mapping of where every piece of code, schema, and configuration file belongs within your /CAN-Do folder layout.

1. Root Directory (/CAN-Do)

- `partitions.csv` — Defines the custom 4MB flash layout (OTA partitions + LittleFS storage partition).
- `README.md` — The master documentation outlining the edge-first architecture, workflow, and build instructions.

2. The Catalog (/CAN-Do/catalog)

- `catalog.json` — The single source of truth containing your entities, state definitions, D1-D8 match masks, and JSON automation rules (like the cold start seat heater example).

3. The Material Design 3 Frontend (/CAN-Do/frontend)

- `main.js` — The client-side `CanDoConnectionManager` that handles WebSocket auto-reconnection, state synchronization via `/api/states`, UI updates, and terminal window log rendering.
- `index.html` & CSS files — Your MD3 dashboard layout, interactive selectors, and visual elements.

4. The ESP-IDF Firmware (/CAN-Do/firmware/main)

- `main.cpp` — The core application loop containing:
 - `app_main()` initialization sequence (NVS, LittleFS, TWAI/CAN driver, queues, and task spawns).
 - The CAN RX and CAN TX FreeRTOS tasks.
 - The **State Cache** (`std::unordered_map`) and **Condition Evaluator**.
 - The **ESP-IDF Logger Hook** (`esp_log_set_vprintf`) that pipes system logs and crashes to the WebSocket terminal.

- The HTTP server routing setup (`start_webserver`).
- **api.cpp** (or grouped inside your server files) — Contains the REST and WebSocket endpoints:
 - `/api/states` (fetches current RAM states for frontend sync).
 - `/api/command` (pushes manual dashboard commands into the FreeRTOS CAN queue).
 - `/api/upload` (handles raw file uploads to LittleFS with the X-File-Path header).
 - `/api/test_automation` (handles dry runs and live fire testing for your automation rules).
 - `/ws` (the WebSocket connection upgrade handler and async broadcaster).
 - `/api/ota` (handles binary firmware uploads for A/B partition flashing).
- **parser.cpp** (or your boot parser file) — Contains the cJSON logic that reads `catalog.json` from LittleFS at startup, translates 1-based indexing (D1-D8) into 0-indexed C++ arrays, and flips the appropriate bits for `match_mask`.
- **CMakeLists.txt** — The build script containing the custom `copy_fs_files` target that automatically stages your `/catalog` and `/frontend` folders and packs them into the LittleFS binary image using `littlefs_create_partition_image`.
- **idf_component.yml** — Declares project dependencies, specifically pulling in the `joltwallet/littlefs` component from the ESP-IDF Component Registry.

Would you like to map out how any specific folder or file links together before you start dropping code into your editor?

User prompt: Yes, that would be helpful actually

Response: Here is how the core files and functions link together across your monorepo so you can trace the data flow from a button press all the way to the CAN bus.

1. The Initialization & Boot Flow (`firmware/main/main.cpp`)

When you power on the ESP32, `app_main()` initializes the system in a strict, safe order:

- Mounts LittleFS and calls the cJSON parser to read `/spiffs/catalog/catalog.json`.
- Converts your human-readable D1-D8 keys into ultra-fast 0-indexed C++ arrays and bitmasks.
- Starts the TWAI (CAN) driver and initializes the `can_state_cache` (`std::unordered_map`).

- Spawns the FreeRTOS **RX Task** (high priority) to listen to the bus and the **TX Task** (normal priority) to process command queues.
- Starts the HTTP/WebSocket server and activates the `esp_log_set_vprintf` hook to pipe system logs straight to your browser.

2. The Live Execution Loop (CAN ⇌ RAM ⇌ Dashboard)

Once running, the components operate independently without blocking network tasks:

- **The State Cache:** Every incoming CAN frame captured by the RX task instantly updates `can_state_cache` in RAM.
- **The Condition Evaluator:** When a trigger fires (like ignition turning on), the engine checks the RAM cache using your defined operators (`less_than`, `equal`, etc.).
- **The WebSocket Broadcaster:** When a state changes, `broadcast_ws_state()` sends a tiny JSON payload to all connected browser tabs, instantly updating your Material Design 3 UI elements (`<md-outlined-select>`, etc.) without a page refresh.

3. The Web Dashboard & OTA Management (firmware/main/api.cpp & frontend/main.js)

The HTTP endpoints tie your browser interface directly into the ESP32's internal queues:

- `/api/command` & `/ws`: Handlers that take manual taps from your phone and drop them into the FreeRTOS TX queue.
- `/api/upload`: Receives raw files sent with the X-File-Path header, allowing you to update frontend HTML/JS or drop a modified `catalog.json` straight onto LittleFS.
- `/api/test_automation`: Executes dry runs (testing math logic against the RAM cache) or live-firing sequences on demand.
- `/api/ota`: Handles binary firmware chunks to flash the inactive OTA partition (`ota_0` / `ota_1`) for wireless updates.

Would you like to start outlining the structure for your first script or testing one of these integration files?
